

05 — Schema Design: E-Commerce Platform

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Domain Overview](#)
- 3. [ASCII ER Diagram](#)
- 4. [Entity Descriptions](#)
- 5. [Complete DDL Script](#)
- 6. [Key Design Decisions](#)
- 7. [Common Queries](#)
- 8. [Performance Indexes](#)
- 9. [Common Mistakes](#)
- 10. [Best Practices](#)
- 11. [Interview Questions & Answers](#)
- 12. [Exercises with Solutions](#)
- 13. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

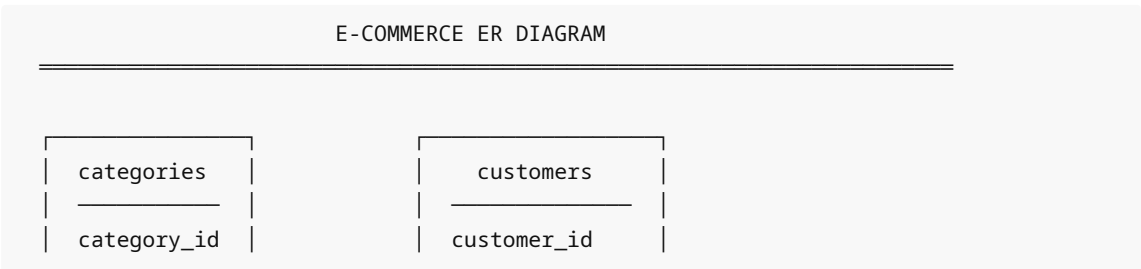
- Design a production-quality e-commerce schema from scratch
- Understand catalog, inventory, ordering, and payment entity design
- Apply normalization and strategic denormalization correctly
- Handle pricing, promotions, and multi-currency considerations
- Write efficient queries for the most common e-commerce operations

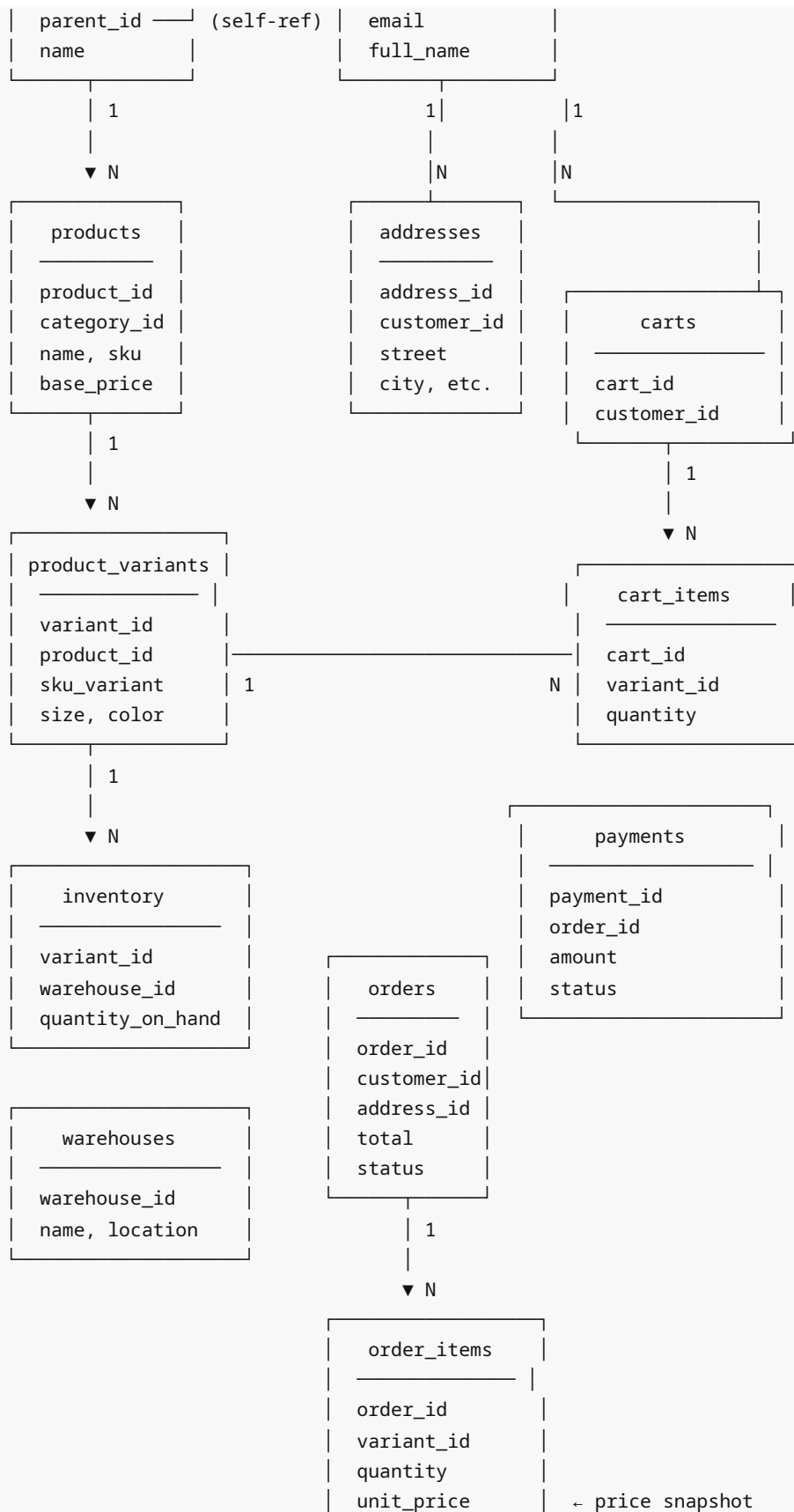
Domain Overview

An e-commerce platform manages:

- **Catalog:** categories, products, product variants (size/color), images
- **Customers:** accounts, addresses, saved payment methods
- **Cart:** shopping sessions with items
- **Orders:** placed orders with items, pricing snapshots
- **Inventory:** stock tracking per variant per warehouse
- **Payments:** payment processing records
- **Promotions:** discount codes, rules

ASCII ER Diagram





Entity Descriptions

Entity	Purpose	Key Points
categories	Product taxonomy	Self-referencing tree
products	Product catalog	Base price, not variant price
product_variants	Size/color variants	Each has its own SKU and price
product_images	Images for products	Ordered, one is primary
customers	Customer accounts	Separated from addresses
addresses	Shipping/billing addresses	Multiple per customer
carts	Shopping sessions	One active cart per customer
cart_items	Items in a cart	Quantity and variant
orders	Placed orders	Immutable after placement
order_items	Line items in an order	Price snapshot!
inventory	Stock levels	Per variant per warehouse
warehouses	Physical locations	For multi-warehouse support
payments	Payment records	One order can have multiple payments
promotions	Discount codes	Rules-based discounts
order_promotions	Applied promotions	M:N: order uses promotion

Complete DDL Script

```
-- =====
-- E-COMMERCE SCHEMA
-- =====

-- EXTENSIONS
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- =====
-- PRODUCT CATALOG
-- =====

CREATE TABLE categories (
    category_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```

    name          VARCHAR(100)    NOT NULL,
    slug           VARCHAR(100)    NOT NULL UNIQUE,
    description     TEXT,
    parent_id      SMALLINT        REFERENCES categories(category_id) ON DELETE SET
NULL,
    display_order  SMALLINT        NOT NULL DEFAULT 0,
    is_active      BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT no_self_parent CHECK (parent_id != category_id)
);

```

```

CREATE INDEX idx_categories_parent ON categories (parent_id);

```

```

CREATE INDEX idx_categories_slug  ON categories (slug);

```

```

COMMENT ON TABLE categories IS 'Product taxonomy tree. parent_id IS NULL for root
categories.';

```

```

CREATE TABLE products (
    product_id      INTEGER        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id     SMALLINT        NOT NULL REFERENCES categories(category_id),
    sku_base        VARCHAR(50)     NOT NULL UNIQUE,
    name            VARCHAR(255)    NOT NULL CHECK (length(trim(name)) >= 2),
    slug            VARCHAR(255)    NOT NULL UNIQUE,
    description      TEXT,
    base_price      NUMERIC(12,4)   NOT NULL CHECK (base_price >= 0),
    cost_price      NUMERIC(12,4)   CHECK (cost_price >= 0),
    brand           VARCHAR(100),
    weight_grams    INTEGER        CHECK (weight_grams > 0),
    attributes      JSONB          NOT NULL DEFAULT '{}',
    search_vec      TSVECTOR        GENERATED ALWAYS AS (
        setweight(to_tsvector('english', name), 'A') ||
        setweight(to_tsvector('english', coalesce(description, '')),
'B')
    ) STORED,
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

```

```

CREATE INDEX idx_products_category ON products (category_id);

```

```

CREATE INDEX idx_products_search  ON products USING GIN (search_vec);

```

```

CREATE INDEX idx_products_attrs   ON products USING GIN (attributes);

```

```

CREATE INDEX idx_products_active  ON products (created_at DESC) WHERE is_active =
TRUE;

```

```

CREATE TABLE product_variants (
    variant_id      INTEGER        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id      INTEGER        NOT NULL REFERENCES products(product_id) ON

```

```

DELETE CASCADE,
    sku                VARCHAR(60)        NOT NULL UNIQUE,
    name               VARCHAR(100)       NOT NULL,    -- e.g., "Red / Large"
    price_modifier     NUMERIC(12,4)      NOT NULL DEFAULT 0, -- +/- from base_price
    price_override     NUMERIC(12,4)      CHECK (price_override >= 0), -- NULL = use
base+modifier
    barcode           VARCHAR(50)        UNIQUE,
    attributes         JSONB              NOT NULL DEFAULT '{}', --
{"color":"red","size":"L"}
    is_active         BOOLEAN             NOT NULL DEFAULT TRUE,
    created_at         TIMESTAMPTZ        NOT NULL DEFAULT now()
);

CREATE INDEX idx_variants_product ON product_variants (product_id);
CREATE INDEX idx_variants_sku    ON product_variants (sku);
CREATE INDEX idx_variants_attrs  ON product_variants USING GIN (attributes);

COMMENT ON COLUMN product_variants.price_override IS 'If set, overrides base_price +
price_modifier completely.';

-- -----

CREATE TABLE product_images (
    image_id          INTEGER              GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id        INTEGER              NOT NULL REFERENCES products(product_id) ON
DELETE CASCADE,
    variant_id        INTEGER              REFERENCES product_variants(variant_id) ON
DELETE SET NULL,
    url               TEXT                 NOT NULL,
    alt_text          VARCHAR(255),
    display_order     SMALLINT             NOT NULL DEFAULT 0,
    is_primary        BOOLEAN              NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMPTZ          NOT NULL DEFAULT now()
);

CREATE INDEX idx_images_product ON product_images (product_id, display_order);

-- =====
-- CUSTOMERS & ADDRESSES
-- =====

CREATE TABLE customers (
    customer_id       BIGINT               GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email             VARCHAR(255)         NOT NULL UNIQUE CHECK (email ~
'^[^\s]+@[^\s]+\.[^\s]+$'),
    password_hash     VARCHAR(100)        NOT NULL,
    first_name        VARCHAR(100)        NOT NULL,
    last_name         VARCHAR(100)        NOT NULL,
    phone             VARCHAR(20)          UNIQUE,
    date_of_birth     DATE,
    is_active         BOOLEAN              NOT NULL DEFAULT TRUE,
    email_verified     BOOLEAN              NOT NULL DEFAULT FALSE,

```

```

        created_at      TIMESTAMPTZ      NOT NULL DEFAULT now(),
        last_login      TIMESTAMPTZ
    );

CREATE INDEX idx_customers_email ON customers (email);
CREATE INDEX idx_customers_name  ON customers (last_name, first_name);

-- =====

CREATE TABLE addresses (
    address_id      INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id     BIGINT       NOT NULL REFERENCES customers(customer_id) ON
DELETE CASCADE,
    label          VARCHAR(50)   NOT NULL DEFAULT 'Home', -- Home, Work, etc.
    first_name     VARCHAR(100)  NOT NULL,
    last_name      VARCHAR(100)  NOT NULL,
    company        VARCHAR(100),
    address_line1  TEXT          NOT NULL,
    address_line2  TEXT,
    city           TEXT          NOT NULL,
    state_province TEXT,
    postal_code    VARCHAR(20)   NOT NULL,
    country_code   CHAR(2)       NOT NULL DEFAULT 'US',
    phone          VARCHAR(20),
    is_default     BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE INDEX idx_addresses_customer ON addresses (customer_id);

-- =====
-- INVENTORY & WAREHOUSES
-- =====

CREATE TABLE warehouses (
    warehouse_id     SMALLINT     GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name             VARCHAR(100)  NOT NULL UNIQUE,
    address_line1    TEXT          NOT NULL,
    city             TEXT          NOT NULL,
    country_code     CHAR(2)       NOT NULL,
    is_active        BOOLEAN       NOT NULL DEFAULT TRUE
);

CREATE TABLE inventory (
    variant_id       INTEGER      NOT NULL REFERENCES
product_variants(variant_id),
    warehouse_id     SMALLINT     NOT NULL REFERENCES warehouses(warehouse_id),
    quantity_on_hand INTEGER      NOT NULL DEFAULT 0 CHECK (quantity_on_hand >=
0),
    quantity_reserved INTEGER     NOT NULL DEFAULT 0 CHECK (quantity_reserved >=
0),
    reorder_level    INTEGER      NOT NULL DEFAULT 5,

```

```

        updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
        PRIMARY KEY (variant_id, warehouse_id),
        CONSTRAINT reserved_le_onhand CHECK (quantity_reserved <= quantity_on_hand)
    );

CREATE VIEW inventory_available AS
SELECT
    variant_id,
    warehouse_id,
    quantity_on_hand - quantity_reserved AS quantity_available
FROM inventory;

-- =====
-- SHOPPING CART
-- =====

CREATE TABLE carts (
    cart_id          UUID          DEFAULT gen_random_uuid() PRIMARY KEY,
    customer_id      BIGINT        REFERENCES customers(customer_id) ON DELETE
CASCADE,
    session_id       VARCHAR(100), -- for guest carts
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    expires_at       TIMESTAMPTZ   NOT NULL DEFAULT now() + INTERVAL '30 days',
    CONSTRAINT cart_has_owner CHECK (
        (customer_id IS NOT NULL) OR (session_id IS NOT NULL)
    )
);

CREATE INDEX idx_carts_customer ON carts (customer_id) WHERE customer_id IS NOT
NULL;
CREATE INDEX idx_carts_session  ON carts (session_id) WHERE session_id IS NOT NULL;

CREATE TABLE cart_items (
    cart_id          UUID          NOT NULL REFERENCES carts(cart_id) ON DELETE
CASCADE,
    variant_id       INTEGER       NOT NULL REFERENCES
product_variants(variant_id),
    quantity         INTEGER       NOT NULL DEFAULT 1 CHECK (quantity > 0),
    added_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (cart_id, variant_id)
);

-- =====
-- PROMOTIONS
-- =====

CREATE TABLE promotions (
    promotion_id     INTEGER       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code             VARCHAR(50)   NOT NULL UNIQUE,
    description       TEXT,
    discount_type     TEXT         NOT NULL CHECK (discount_type IN

```

```

('percentage','fixed_amount','free_shipping')),
    discount_value NUMERIC(10,4) NOT NULL CHECK (discount_value > 0),
    minimum_order NUMERIC(12,4) NOT NULL DEFAULT 0,
    max_uses INTEGER, -- NULL = unlimited
    uses_count INTEGER NOT NULL DEFAULT 0,
    max_uses_per_customer INTEGER NOT NULL DEFAULT 1,
    starts_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    ends_at TIMESTAMPTZ,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT valid_percentage CHECK (
        discount_type != 'percentage' OR (discount_value BETWEEN 0.01 AND 100)
    )
);

-- =====
-- ORDERS
-- =====

CREATE TABLE orders (
    order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_number TEXT NOT NULL UNIQUE DEFAULT
        'ORD-' || to_char(now(), 'YYYYMMDD') || '-' ||
        lpad(nextval('orders_order_number_seq')::TEXT, 6, '0'),
    customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
    shipping_address_id INTEGER REFERENCES addresses(address_id) ON DELETE SET
NULL,
    billing_address_id INTEGER REFERENCES addresses(address_id) ON DELETE SET
NULL,

    -- Status tracking
    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN
('pending','confirmed','processing','shipped',
'delivered','cancelled','refunded')),

    -- Financial snapshot (immutable after placement)
    subtotal NUMERIC(12,4) NOT NULL CHECK (subtotal >= 0),
    discount_amount NUMERIC(12,4) NOT NULL DEFAULT 0 CHECK (discount_amount >= 0),
    shipping_amount NUMERIC(12,4) NOT NULL DEFAULT 0 CHECK (shipping_amount >= 0),
    tax_amount NUMERIC(12,4) NOT NULL DEFAULT 0 CHECK (tax_amount >= 0),
    total_amount NUMERIC(12,4) NOT NULL CHECK (total_amount >= 0),

    -- Timestamps
    ordered_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    confirmed_at TIMESTAMPTZ,
    shipped_at TIMESTAMPTZ,
    delivered_at TIMESTAMPTZ,
    cancelled_at TIMESTAMPTZ,

    notes TEXT,
    currency_code CHAR(3) NOT NULL DEFAULT 'USD',
    CONSTRAINT total_matches CHECK (

```

```

        total_amount = subtotal - discount_amount + shipping_amount + tax_amount
    )
);

CREATE SEQUENCE orders_order_number_seq START WITH 1;

CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_orders_status ON orders (status, ordered_at DESC);
CREATE INDEX idx_orders_date ON orders (ordered_at DESC);

-- -----

CREATE TABLE order_items (
    order_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE
CASCADE,
    variant_id INTEGER NOT NULL REFERENCES
product_variants(variant_id),

    -- Snapshot of product info at time of order (intentional denormalization)
    product_name TEXT NOT NULL,
    variant_name TEXT NOT NULL,
    sku VARCHAR(60) NOT NULL,

    -- Financial snapshot
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(12,4) NOT NULL CHECK (unit_price >= 0),
    discount_amount NUMERIC(12,4) NOT NULL DEFAULT 0 CHECK (discount_amount >= 0),
    line_total NUMERIC(12,4) GENERATED ALWAYS AS (
        quantity * unit_price - discount_amount
    ) STORED
);

CREATE INDEX idx_order_items_order ON order_items (order_id);
CREATE INDEX idx_order_items_variant ON order_items (variant_id);

-- -----

CREATE TABLE order_promotions (
    order_id BIGINT NOT NULL REFERENCES orders(order_id),
    promotion_id INTEGER NOT NULL REFERENCES promotions(promotion_id),
    discount_applied NUMERIC(12,4) NOT NULL,
    applied_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (order_id, promotion_id)
);

-- =====
-- PAYMENTS
-- =====

CREATE TABLE payments (
    payment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```

    order_id          BIGINT          NOT NULL REFERENCES orders(order_id),
    amount            NUMERIC(12,4)   NOT NULL CHECK (amount > 0),
    currency_code     CHAR(3)         NOT NULL DEFAULT 'USD',
    payment_method    TEXT            NOT NULL CHECK (payment_method IN
('card','paypal','bank_transfer','wallet')),
    status            TEXT            NOT NULL DEFAULT 'pending'
                        CHECK (status IN
('pending','processing','completed','failed','refunded')),
    gateway           TEXT,           -- Stripe, Braintree, etc.
    gateway_tx_id     TEXT,           -- External transaction ID
    processed_at      TIMESTAMPTZ,
    created_at        TIMESTAMPTZ     NOT NULL DEFAULT now()
);

CREATE INDEX idx_payments_order ON payments (order_id);

```

Key Design Decisions

1. **Product variants as a separate table** — Each SKU has its own row in `product_variants`. This cleanly handles size/color combinations without complex product attribute schemas.
2. **Price snapshot in order_items** — `unit_price` is stored at order time. Product prices change; order history must not.
3. **Denormalized product info in order_items** — `product_name`, `variant_name`, `sku` are copied at order time. Deleting/renaming a product must not break order history.
4. **Generated column for line_total** — Always consistent, always computed, indexed if needed.
5. **Cart uses UUID PK** — Cart IDs are exposed to the client (cookies). UUIDs are non-guessable.
6. **Addresses are snapshots** — Shipping/billing address IDs point to the addresses table, but the address rows should be treated as immutable once an order is placed.
7. **Financial total constraint** — `total_amount = subtotal - discount + shipping + tax` is enforced as a CHECK constraint.

Common Queries

```

-- Get order details with all line items
SELECT
    o.order_number,
    o.status,
    o.total_amount,
    c.email AS customer_email,
    oi.product_name,
    oi.variant_name,
    oi.quantity,
    oi.unit_price,
    oi.line_total

```

```

FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_id = 12345;

-- Check available inventory
SELECT
    pv.sku,
    pv.name,
    sum(i.quantity_on_hand - i.quantity_reserved) AS available_total
FROM product_variants pv
JOIN inventory i ON i.variant_id = pv.variant_id
WHERE pv.product_id = 42
GROUP BY pv.variant_id, pv.sku, pv.name;

-- Top-selling products (last 30 days)
SELECT
    p.name,
    sum(oi.quantity) AS units_sold,
    sum(oi.line_total) AS revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN product_variants pv ON pv.variant_id = oi.variant_id
JOIN products p ON p.product_id = pv.product_id
WHERE o.ordered_at >= now() - INTERVAL '30 days'
    AND o.status NOT IN ('cancelled', 'refunded')
GROUP BY p.product_id, p.name
ORDER BY revenue DESC
LIMIT 10;

-- Customer lifetime value
SELECT
    c.customer_id,
    c.email,
    count(DISTINCT o.order_id) AS total_orders,
    sum(o.total_amount) AS lifetime_value,
    avg(o.total_amount) AS avg_order_value,
    max(o.ordered_at) AS last_order_date
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status != 'cancelled'
GROUP BY c.customer_id, c.email
ORDER BY lifetime_value DESC NULLS LAST;

```

Performance Indexes

```

-- Full-text product search
CREATE INDEX idx_products_fts ON products USING GIN (search_vec);

-- Category browse with price range
CREATE INDEX idx_products_category_price ON products (category_id, base_price)

```

```
WHERE is_active = TRUE;

-- Orders by customer (most recent first)
CREATE INDEX idx_orders_customer_date ON orders (customer_id, ordered_at DESC);

-- Inventory low stock alert
CREATE INDEX idx_inventory_low_stock ON inventory (variant_id)
WHERE quantity_on_hand - quantity_reserved < reorder_level;

-- Active promotions lookup
CREATE INDEX idx_promotions_code ON promotions (code)
WHERE is_active = TRUE AND (ends_at IS NULL OR ends_at > now());
```

Common Mistakes

1. **Storing only product_id in order_items** (no price/name snapshot) — Product changes break order history.
 2. **One address per customer** — Customers have work/home/gift addresses.
 3. **No inventory table** — Stock levels must be tracked separately from products.
 4. **Status as integer** — Use meaningful text strings with CHECK constraints.
 5. **No total constraint** — Allows total_amount to be set inconsistently.
-

Interview Questions & Answers

Q1: Why do we store product_name and unit_price in order_items instead of just the variant_id?

A: Because products and prices change. An order is a historical record — it must reflect what the customer actually paid and what they ordered. If we only stored `variant_id`, renaming the product or changing its price would retroactively alter all past orders. The snapshot approach is intentional and correct.

Q2: How would you handle a product with multiple prices (different currencies)?

A: Add a `product_prices` table: `(variant_id, currency_code, price)`. Orders capture the price in the order's currency. The `base_price` column on products can remain the "canonical" currency price.

Q3: How would you design flash sales or time-limited pricing?

A: Add a `price_rules` table with `(variant_id, price, starts_at, ends_at, rule_type)`. A `get_current_price(variant_id)` function checks for active price rules and returns the appropriate price. The checkout process calls this function and stores the result in `order_items.unit_price`.

Q4: How would you handle product search with filters like color and size?

A: Use the `product_variants.attributes` JSONB column with a GIN index. Query: `WHERE pv.attributes @> '{"color": "red"}' AND pv.attributes @> '{"size": "L"}'`. For faceted navigation, aggregate variant attributes across products.

Q5: What happens to cart items if a product is deleted?

A: `cart_items.variant_id` has a FK to `product_variants`. If the variant is deleted, the cart item FK violation prevents deletion. Best practice: soft-delete products (set `is_active = FALSE`) rather than hard-deleting.

Exercises with Solutions

Exercise 1

Write a query to find all products with less than 5 units available across ALL warehouses, including the current total available.

Solution:

```
SELECT
  p.product_id,
  p.name AS product_name,
  pv.sku,
  pv.name AS variant_name,
  sum(i.quantity_on_hand - i.quantity_reserved) AS total_available
FROM products p
JOIN product_variants pv ON pv.product_id = p.product_id
LEFT JOIN inventory i ON i.variant_id = pv.variant_id
WHERE p.is_active = TRUE AND pv.is_active = TRUE
GROUP BY p.product_id, p.name, pv.variant_id, pv.sku, pv.name
HAVING sum(i.quantity_on_hand - i.quantity_reserved) < 5
ORDER BY total_available, p.name;
```

Exercise 2

Write the logic to apply a promotion code at checkout and calculate the discounted total.

Solution:

```
CREATE OR REPLACE FUNCTION apply_promotion(
  p_cart_id UUID,
  p_promo_code TEXT
) RETURNS TABLE (subtotal NUMERIC, discount NUMERIC, new_total NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
  v_promo promotions%ROWTYPE;
  v_subtotal NUMERIC;
BEGIN
  -- Get cart subtotal
  SELECT sum(ci.quantity * coalesce(pv.price_override, p.base_price +
pv.price_modifier))
  INTO v_subtotal
  FROM cart_items ci
  JOIN product_variants pv ON pv.variant_id = ci.variant_id
  JOIN products p ON p.product_id = pv.product_id
  WHERE ci.cart_id = p_cart_id;

  -- Validate promotion
  SELECT * INTO v_promo FROM promotions
  WHERE code = upper(p_promo_code)
  AND is_active = TRUE
```

```

AND starts_at <= now()
AND (ends_at IS NULL OR ends_at > now())
AND (max_uses IS NULL OR uses_count < max_uses)
AND minimum_order <= v_subtotal;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Invalid or expired promotion code: %', p_promo_code;
END IF;

RETURN QUERY SELECT
    v_subtotal,
    CASE v_promo.discount_type
        WHEN 'percentage' THEN round(v_subtotal * v_promo.discount_value / 100,
4)

        WHEN 'fixed_amount' THEN least(v_promo.discount_value, v_subtotal)
        ELSE 0
    END,
    v_subtotal - CASE v_promo.discount_type
        WHEN 'percentage' THEN round(v_subtotal * v_promo.discount_value / 100,
4)

        WHEN 'fixed_amount' THEN least(v_promo.discount_value, v_subtotal)
        ELSE 0
    END;
END;
$$;

```

Cross-References

- `04_entity_relationships.md` — all relationship types used here
- `06_schema_design_banking.md` — payment processing schema
- `10_design_patterns.md` — audit trail, soft deletes used in this schema
- `../05_PostgreSQL_Core/05_json_jsonb.md` — product attributes in JSONB
- `../07_Indexes/` — index strategy for this schema